

Async Credit Check Architecture

Qred Card Application — Technical Overview

1. Overview

When a company applies for a card, the credit check runs **asynchronously**. The API returns **202 Accepted** immediately with a *requestId*. The client then polls *GET /cards/credit-check/:requestId/status* until the result is ready. This avoids blocking HTTP threads while waiting on external Swedish credit bureaus (UC, CreditSafe, Kronofogden), which can take 1–10 seconds to respond.

2. Message Queue — AWS SNS + SQS

The card service publishes a **CreditCheckRequestedEvent** to an **SNS topic** (*credit-check-requested*). SNS fans the message out to provider-specific **SQS queues**. Each queue has a dedicated worker that calls the appropriate bureau. SQS handles retries automatically (up to 3 attempts) and moves failed messages to a **Dead Letter Queue (DLQ)** for manual inspection.

```
SNS topic: credit-check-requested
■■■ SQS queue: credit-check-uc
■■■ SQS queue: credit-check-creditsafe
■■■ SQS queue: credit-check-kronofogden
```

3. CreditCheckFactory Pattern

A factory selects the correct provider at runtime based on check type. All providers implement a common **ICreditCheckProvider** interface, making them interchangeable and easy to mock in tests.

```
interface ICreditCheckProvider {
  check(input: CreditCheckInput): Promise<CreditCheckResult>;
}

class CreditCheckFactory {
  static create(type: 'uc' | 'creditsafe' | 'kronofogden' | 'internal')
    : ICreditCheckProvider
}
```

Providers:

UC — Sweden's main credit bureau; checks individual credit history via personnummer.

CreditSafe — Business credit ratings for company-level risk assessment.

Kronofogden — Swedish Enforcement Authority; flags active debt enforcement records.

Internal — Fallback using internal transaction history; used for testing or small limits.

4. Request Flow

1. **POST /companies/:id/cards** — controller calls `CardService.apply()`
2. Service publishes `CreditCheckRequestedEvent` to SNS
3. Returns **202 Accepted** { `requestId`, `status`: 'under_review' }
4. SQS worker picks up event, `CreditCheckFactory` selects provider
5. Provider calls external bureau API
6. Result written to DB: `card.status` → 'active' or 'blocked'
7. Client polls **GET /cards/credit-check/:requestId/status** for result